

Doom  [rspress](#)  Alauda            

□ □ □ □

- □□□□□□□□□□□□□□□□□□
- □□□□□□□□
- □□□□□

MDX

Markdown

React



0000000000 MDX 000 ↗

0000

0000000000 Doom 00

00

00

0000

000000

00

Markdown

Callouts

Mermaid

📄📄📄

MDX

📄 MDX 📄📄📄📄📄📄📄📄📄📄

rspress 📄

doom 📄

📄📄📄📄📄

📄📄

📄📄📄📄📄📄📄📄📄

i18n.json

.ts/.tsx

.mdx

API 📄

📄 API

CRD

📄📄📄

📄 openapi 📄

📄📄📄📄📄

props

📄

📄📄

📄📄

📄📄📄

📄📄📄📄

📄📄📄📄📄

📄📄📄📄📄

📄📄📄

📄📄📄📄📄📄

📄📄📄

📄📄 PDF

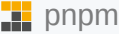
📄📄📄

📄📄📄📄

📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

```
mkdir my-docs && cd my-docs
```

📄 `npm init -y` 📄📄📄📄📄📄📄📄📄📄📄📄📄 `npm`📄`yarn` 📄 `pnpm` 📄 `doom:`



```
npm install -D @alauda/doom typescript
```

创建文档目录:

```
# 创建 docs 目录并初始化
mkdir docs/en && echo '# Hello World' > docs/en/index.md
mkdir docs/zh && echo '# 文档' > docs/zh/index.md
```

在 `package.json` 中添加:

```
{
  "scripts": {
    "dev": "doom dev",
    "build": "doom build",
    "new": "doom new",
    "serve": "doom serve",
    "translate": "doom translate",
    "export": "doom export"
  }
}
```

创建 `doom.config.yml`:

```
title: My Docs
```

创建 `tsconfig.json` 文件

```
{
  "compilerOptions": {
    "jsx": "react-jsx",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "resolveJsonModule": true,
    "skipLibCheck": true,
    "strict": true,
    "target": "ESNext",
  },
  "mdx": {
    "checkMdx": true,
  },
}
```

global.d.ts

```
/// <reference types="@alauda/doom/runtime" />
```

.mdx doom

```
doom -h
```

```
# output
```

```
Usage: doom [options] [command]
```

```
Doctor Doom making docs.
```

```
Options:
```

```
  -V, --version                output the version number
  -c, --config <config>       Specify the path to the config file
  -v <version>                 Specify the version of the
documentation, can also be `unversioned` or `unversioned-x.y`
  -b, --base <base>           Override the base of the
documentation
  -p, --prefix <prefix>       Specify the prefix of the
documentation base
  -f, --force [boolean]        Force to
                                1. fetch latest reference remotes or
                                scaffolding templates, otherwise use local cache
                                2. translate ignore hash equality
check and original text (default: false)
  -i, --ignore [boolean]       Ignore internal routes (default:
false)
  -d, --download [boolean]     Display download pdf link on nav bar
(default: false)
  -e, --export [boolean]       Run or build in exporting PDF mode,
`apis/**` and `*/apis/**` routes will be ignored automatically (default:
false)
  -I, --include <language...> Include only the specific
language(s), `en ru` for example
  -E, --exclude <language...> Include all languages except the
specific language(s), `ru` for example
  -o, --out-dir <path>         Override the `outDir` defined in the
config file or the default `dist/{base}/{version}`, the resulting path
will be `dist/{outDir}/{version}`
  -r, --redirect <enum>        Whether to redirect to the locale
closest to `navigator.language` when the user visits the site, could be
`auto`, `never` or `only-default-lang` (default: "only-default-lang")
  -R, --edit-repo [boolean|url|lang] Whether to enable or override the
`editRepoBaseUrl` config feature, `https://github.com/` prefix could be
omitted; if a language code is provided, all other languages will use the
same repo but with the language code as sub-path (default: false)
  -a, --algolia [boolean|alauda] Whether to enable or use the alauda
```

```
(docs.alauda.io) preset for Algolia search (default: false)
  -S, --site-url           Whether to enable the siteUrl for
sitemap generation (default: false)
  -n, --no-open           Do not open the browser after
starting the server
  --lang <language>      Specify the language of the
documentation
  -h, --help              display help for command
```

Commands:

```
dev [options] [root]      Start the development server
build [root]              Build the documentation
preview|serve [options] [root] Preview the built documentation
new [template]            Generate scaffolding from templates
translate [options] [root] Translate the documentation
export [options] [root]   Export the documentation as PDF,
`apis/**` and `*/apis/**` routes will be ignored automatically
lint [options] [root]     Lint the documentation
help [command]            display help for command
```

yarn dev

```
doom dev -h
```

```
# output
```

```
Usage: doom dev [options] [root]
```

```
Start the development server
```

```
Arguments:
```

```
  root                                Root directory of the documentation
```

```
Options:
```

```
  -H, --host [host]                  Dev server host name
  -P, --port [port]                  Dev server port number
  -l, --lazy [boolean]               Whether to enable `lazyCompilation` which
could improve the compilation performance (default: true)
  --no-lazy                          Do not enable `lazyCompilation`
  -h, --help                         display help for command
```

📄📄📄📄📄

📄 yarn build 📄📄📄📄📄📄📄📄📄📄📄📄📄 dist 📄📄📄📄📄

📄📄📄📄

📄 yarn serve 📄📄📄📄📄📄📄📄📄📄📄📄📄 -b , -p 📄📄📄📄📄📄📄📄📄📄 -b , -p 📄
📄

📄📄📄📄📄📄

📄 yarn new 📄📄📄📄📄📄📄📄📄📄📄📄📄

📄📄📄📄

```
doom translate -h
```

```
# output
```

```
Usage: doom translate [options] [root]
```

Translate the documentation

Arguments:

root Root directory of the documentation

Options:

-s, --source <language> Document source language, one of en, zh, ru (default: "en")

-t, --target <language> Document target language, one of en, zh, ru (default: "zh")

-g, --glob <path...> Glob patterns of source dirs/files to translate

-C, --copy [boolean] Wether to copy relative assets to the target directory instead of following links (default: false)

-h, --help display help for command

- `-g, --glob` 指定要翻译的源目录/文件的glob模式 `glob` 模式支持通配符 `*` 和 `?`。
 1. `yarn translate -g abc xyz` 将 `<root>/<source>/abc` , `<root>/<source>/xyz` 复制到 `<root>/<target>/abc` , `<root>/<target>/xyz` 。
 2. `yarn translate -g '*'` 将 `<root>/<source>` 下的所有文件复制到 `<root>/<target>` 。
- `-C, --copy` 指定是否将源目录/文件中的相对资产复制到目标目录，而不是通过链接。默认为 `false`。
 - 当 `-C` 为 `true` 时：
 1. `/<source>/abc.jpg` 复制到 `<root>/public/<source>/abc.jpg` 。
 2. `<root>/<source>/abc.mdx` 复制到 `./assets/xyz.jpg` 。
 3. `<root>/<source>/assets/xyz.jpg` 复制到 `<root>/<target>/assets/xyz.jpg` 。
 - 当 `-C` 为 `false` 时：
 1. `/<source>/abc.jpg` 复制到 `<root>/public/<source>/abc.jpg` 。
 2. `<root>/<source>/assets/xyz.jpg` 复制到 `<root>/<source>/assets/xyz.jpg` 。
 3. `<root>/<source>/assets/xyz.jpg` 复制到 `<root>/<target>/assets/xyz.jpg` 。

```
<root>/<target>/assets/<target>/xyz.jpg
./assets/<target>/xyz.jpg
```

- `internalRoutes`
 1. `/<source>/abc.jpg` → `<root>/public/<target>/abc.jpg` → `/<target>/abc.jpg`
 2. `<root>/<source>/abc.mdx` → `./assets/<source>/xyz.jpg` → `<root>/<target>/assets/<target>/xyz.jpg` → `./assets/<target>/xyz.jpg` → `../<source>/assets/<target>/xyz.jpg`

WARNING

`-g '*'` → `source` → `target` → `internalRoutes` → `target`

TIP

`AZURE_OPENAI_API_KEY` → Leader

```
il8n:
  title:
    en: DevOps Connectors
  additionalPrompts: 'Connectors'
  disableAutoTranslation: false
title: DevOps
```

PDF

WARNING

📄📄📄📄📄📄📄 yarn build 📄📄

doom export -h

output

Usage: doom export [options] [root]

Export the documentation as PDF, `apis/\*\*` and `\*/apis/\*\*` routes will be ignored automatically

Arguments:

root Root directory of the documentation

Options:

-H, --host [host] Serve host name
 -P, --port [port] Serve port number (default: "4173")
 -h, --help display help for command

📄 yarn export 📄📄📄 PDF 📄📄📄📄📄📄 -b , -p 📄📄📄📄📄📄📄 -b , -p
 📄

📄📄📄📄 playwright ↗📄📄📄📄📄 build-harbor.alauda.cn/frontend/playwright-runner:doom 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

.env.yarn

PLAYWRIGHT_DOWNLOAD_HOST="https://cdn.npmmirror.com/binaries/playwright"

📄📄📄📄📄📄 pdf 📄📄📄 doom 📄📄📄📄📄📄📄📄 pdf 📄📄📄📄📄📄📄📄📄📄

📄📄📄📄

```
doom lint -h
```

```
# output
```

```
Usage: doom lint [options] [root]
```

```
Lint the documentation
```

```
Arguments:
```

```
  root                Root directory of the documentation
```

```
Options:
```

```
  -g, --glob <path...>  Glob patterns of source dirs/files to lint
                          (default: "**/*.js,jsx,ts,tsx,md,mdx")
  --no-cspell            Disable cspell linting
  -h, --help            display help for command
```

```
doom lint ESLint cspell
```

[ESLint](#) / [CSpell](#):

- `eslint.config.mjs`:

```
import doom from '@alauda/doom/eslint'

export default doom(new URL('docs', import.meta.url))
```

- `cspell.config.mjs`:

```
export { default } from '@alauda/doom/cspell'
```

`CWD` `.cspell` `CSpell`
`.cspell/k8s.txt`

```
k8s
kubernetes
```



📄

📄 `doom` 📄📄

📄📄

📄📄

API 📄📄

📄📄📄📄📄

📄📄📄

📄📄📄

📄📄📄

📄📄📄📄📄

📄📄📄📄📄📄

📄📄📄📄📄

`sites.yaml` 📄

📄📄

📄📄📄📄📄📄

📄📄📄

📄📄📄

Algolia 📄📄

Sitemap 📄

📄

📄“📄📄📄📄”📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄

📄📄📄

📄📄

📄

📄

Markdown

Callouts

Mermaid

📄📄📄

MDX

📄 MDX 📄📄📄📄📄📄📄📄📄📄

rspress 📄

doom 📄

📄📄📄📄📄

📄📄

📄📄📄📄📄📄📄📄

i18n.json

.ts/.tsx

.mdx

API 00

00 API

CRD

0000

00 openapi 00

000000

props

00

0000

000000

00

000000000000000000000000 ACP 00

00000

00000

000000

00000000

📄📄

📄📄

📄📄📄

📄📄📄

API 📄📄📄

📄📄📄📄📄📄

📄📄📄📄📄

frontmatterMode

📄📄📄📄📄

📄📄📄📄

📄📄📄📄📄📄

📄📄📄📄📄📄📄

📄📄📄📄📄📄

sites.yaml 📄📄

📄📄📄📄

📄📄📄📄📄📄📄

📄📄📄📄📄

📄📄📄📄📄

Algolia 📄📄📄📄

Sitemap 📄📄

📄📄📄📄

[illegible]

```
js/ts @alauda/doom/config defineConfig
```

```
import { defineConfig } from '@alauda/doom/config'

export default defineConfig({})
```

--	--	--	--

- `lang` 指定する言語 `en` 指定しない場合は `null` または `undefined`
- `title` タイトル
- `logo` 指定する `logo` のパス `public` フォルダに存在するかどうかを確認する `doom` は `alauda logo`
- `logoText` `logo` のテキスト
- `icon` ファビコンのパス `logo`
- `base` ベースパス `product-docs` の場合は `/`
- `outDir` 出力ディレクトリ `dist/{base}/{version}` または `dist/{outDir}/{version}` `version` はバージョン番号

API ☐ ☐ ☐ ☐

api:

```
# CRD 资源定义文件 doom.config.* 资源定义文件 glob 资源定义文件
```

crds:

```
- docs/shared/crds/*.yaml
```

```
# OpenAPI 资源定义文件 doom.config.* 资源定义文件 glob 资源定义文件
```

openapis:

```
- docs/shared/openapis/*.json
```

```
# 为 openapi 资源定义文件提供资源定义文件
```

```
# 为
```

<https://doom.alauda.cn/apis/references/CodeQuality.html#v1alpha1.CodeQualityS>

references:

```
v1alpha1.CodeQualityBranch:
```

</apis/references/CodeQualityBranch#v1alpha1.CodeQualityBranch>

```
# 为 API 资源定义文件提供 gateway 资源定义文件
```

```
pathPrefix: /apis
```

资源定义文件 API 资源

资源定义文件

```
# 资源定义文件 doom.config.* 资源定义文件 glob 资源定义文件
```

permission:**functionresources:**

```
# `kubectl get functionresources`
```

```
- docs/shared/functionresources/*.yaml
```

roletemplates:

```
# `kubectl get roletemplates -l
```

```
auth.cpaas.io/roletemplate.official=true`
```

```
- docs/shared/roletemplates/*.yaml
```

资源定义文件

资源定义文件

reference:

```

- repo: alauda-public/product-doc-guide #
branch: # [string]
publicBase: # [string] /images/xx.png
docs/public
sources:
- name: anchor #
path: docs/index.mdx# #
doom.config.*
ignoreHeading: # [boolean] true
processors: #
- type: ejsTemplate
data: # ejs `<%= data.xx %>`
frontmatterMode: merge # frontmatter ignore
ignore/merge/replace/remove

```

frontmatterMode

- ignore** frontmatter frontmatter
- merge** frontmatter key
- replace** frontmatter frontmatter
- remove** frontmatter

releaseNotes:**queryTemplates:**

fixed: # ejs jql

unfixed:

release-notes.md

```
<!-- release-notes-for-bugs?template=fixed&project=DevOps -->
```

release-notes.mdx

```
{/* release-notes-for-bugs?template=fixed&project=DevOps */}
```

```

template=fixed&project=DevOps  fixed  queryTemplates 
query  project=DevOps  ejc  fixed  jira  jql 
https://jira.alauda.cn/rest/api/2/search?jql=<jql>  API 
JIRA_USERNAME  JIRA_PASSWORD 

```

--	--	--	--	--

```
sidebar:
  collapsed: false # 折りたたみ状態かどうか false
```

--	--	--	--	--	--	--	--

```
internalRoutes: # 忽略 glob 匹配的 docs 下的 cli 子目录 -i, --ignore` 忽略的
子目录/子目录
- '*/internal/**'
```

```
onlyIncludeRoutes: # 只包含 docs 下的 cli 子目录，忽略其他子目录
  - '*/internal/**'
internalRoutes:
  - '*/internal/overview.mdx'
```

--	--	--	--

translate:

```
# 将源语言 `sourceLang` 翻译成目标语言 `targetLang`，`userPrompt`，
`additionalPrompts`，`terms`，`titleTranslationPrompt`
# 将 `sourceLang` 翻译成 `targetLang` 的 `terms` 和 `title` 的翻译
# `userPrompt` 的翻译
# `additionalPrompts` 的翻译 `frontmatter.il18n` 的 `additionalPrompts`
的翻译
# `terms` 和 `titleTranslationPrompt` 的翻译 AI
的翻译
# 的翻译
```

systemPrompt: |

You are a professional technical documentation engineer, skilled in writing high-quality technical documentation in <%= targetLang %>. Please accurately translate the following text from <%= sourceLang %> to <%= targetLang %>, maintaining the style consistent with technical documentation in <%= sourceLang %>.

Baseline Requirements

- Sentences should be fluent and conform to the expression habits of the <%= targetLang %> language.
- Input format is MDX; output format must also retain the original MDX format. Do not translate the names of jsx components such as <Overview />, and do not wrap output in unnecessary code blocks.
- ****CRITICAL****: Do not translate or modify ANY link content in the document. This includes:
 - **URLs in markdown links**: [text](URL) - keep URL exactly as is
 - **Reference-style links**: [text][ref] and [ref]: URL - keep both ref and URL unchanged
 - **Inline URLs**: https://example.com - keep completely unchanged
 - **Image links**: ![alt](src) - keep src unchanged, but alt text can be translated
 - **Anchor links**: [text](#anchor) - keep #anchor unchanged
 - Any href attributes in HTML tags - keep unchanged
- Do not translate professional technical terms and proper nouns, including but not limited to: Kubernetes, Docker, CLI, API, REST, GraphQL, JSON, YAML, Git, GitHub, GitLab, AWS, Azure, GCP, Linux, Windows, macOS, Node.js, React, Vue, Angular, TypeScript, JavaScript, Python, Java, Go, Rust, etc. Keep these terms in their original form.
- The title field and description field in frontmatter should be translated, other frontmatter fields should retain and do not translate.
- Content within MDX components needs to be translated, whereas MDX component names and parameter keys do not.
- Do not modify or translate any placeholders in the format of

`__ANCHOR_N__` (where N is a number). These placeholders must be kept exactly as they appear in the source text.

- Keep original escape characters like backslash, angle brackets, etc. unchanged during translation.
- Do not add any escape characters to special characters like `[]`, `()`, `{}`, etc. unless they were explicitly present in the source text. For example:
 - If source has "Architecture [Optional]", keep it as "Architecture [Optional]" (not "Architecture \[Optional]")
 - If source has "Function (param)", keep it as "Function (param)" (not "Function \((param)")
 - Only add escape characters if they were present in the original text
- Preserve and do not translate the following comments, nor modify their content:
 - `{/* release-notes-for-bugs */}`
 - `<!-- release-notes-for-bugs -->`
- Remove and do not retain the following comments:
 - `{/* reference-start */}`
 - `{/* reference-end */}`
 - `<!-- reference-start -->`
 - `<!-- reference-end -->`
- Ensure the original Markdown format remains intact during translation, such as frontmatter, code blocks, lists, tables, etc.
- Do not translate the content of the code block.

```
<% if (titleTranslationPrompt) { %>
<%- titleTranslationPrompt %>
<% } %>
<% if (terms) { %>
<%- terms %>
<% } %>
```

```
<% if (userPrompt || additionalPrompts) { %>
```

```
## Additional Requirements
```

These are additional requirements for the translation. They should be met along with the baseline requirements, and in case of any conflict, the baseline requirements should take precedence.

The text for translation is provided below, within triple quotes:

```
"""
```

```
<% if (userPrompt) { %>
<%- userPrompt %>
<% } %>
```

```
<% if (additionalPrompts) { %>
<%- additionalPrompts %>
```

```
<% } %>
""
<% } %>
```

📄📄📄📄📄📄📄📄

```
editRepoBaseUrl: alauda/doom/tree/main/docs # https://github.com/ 📄📄📄📄📄
📄📄 ` -R, --edit-repo` 📄📄📄📄📄📄
```

📄📄📄📄📄📄

```
export:
- name: Concepts # 📄📄📄📄 pdf 📄📄📄📄📄📄
  scope: '*/concepts' # 📄📄📄📄📄📄📄📄📄📄 glob 📄📄📄📄 docs 📄
```

📄📄📄📄📄📄

```
lint:
  cspellOptions: # 📄📄cspell 📄📄📄📄
https://github.com/streetsidesoftware/cspell/tree/main/packages/cspell-
eslint-plugin#options
```

Algolia 📄📄📄📄

```
algolia: # 📄📄Algolia 📄📄📄📄📄📄 ` -a, --algolia` 📄📄📄📄📄📄
  appId: # Algolia 📄 ID
  apiKey: # Algolia API Key
  indexName: # Algolia 📄📄📄
```

📄📄 public/robots.txt 📄 Algolia 📄📄📄

INFO

rspress Algolia [🔗](#) @alauda/doom/theme

theme/index.ts

```
export * from '@alauda/doom/theme'
```

Sitemap ☐ ☐

```
siteUrl: https://docs.alauda.cn # 指定 URL 生成 sitemap 文件 -S, --site-  
url` 指定生成 sitemap 文件的 URL
```

□ □ □ □

□ □ □

11

11

--	--	--	--

index

index.md index.mdx

```
├─ index.md
├─ start.mdx
└─ usage
    ├── index.mdx
    └─ convention.md
```

--	--	--	--	--	--

1. `public`
2. `public/_remotes`
`*/public/_remotes` `.gitignore`

frontmatter

□□□□□□□□ MDX □□ .mdx □□□□□□□□ frontmatter □□□□□□□□

```
[] index.md [] index.mdx [] frontmatter []  
weight [] weight []
```

WARNING

index.mdx Overview

00

<Overview />

0000000000

Markdown

000000 [gfm](#) 00000Doom 0000000000 Markdown 000000

00

Callouts

Mermaid

0000

Callouts

0000000

NOTE

- 000000000000000000000000 ; , % , # , // , /** */ , -- 0 <!-- --> 0
- 000000000000000000000000 [\!code callout] 0000
- 0000 :::callouts 000000000000000000000000 <div class="doom-callouts"> 0
<Callouts> 0000

```
```sh
```

```
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
+ 218 MiB \ # [!code callout]
+ 8 MiB × (number of vCPUs) \ # [!code callout]
+ 16 MiB × (number of graphics devices) \ # [!code callout]
+ (additional memory overhead) # [!code callout]
```

```
```
```

```
:::callouts
```

1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
 - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
 - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
 - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

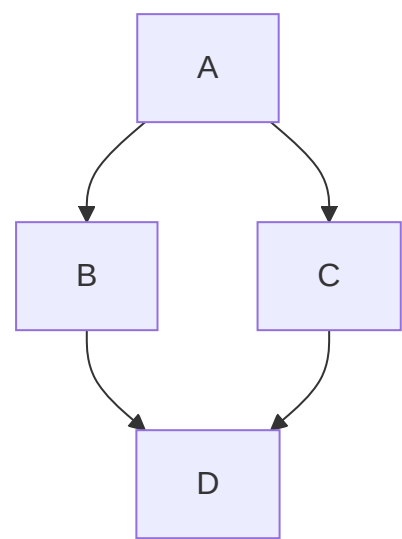
```
:::
```

```
Memory overhead per virtual machine ≈ (1.002 × requested memory) \
+ 218 MiB \ ①
+ 8 MiB × (number of vCPUs) \ ②
+ 16 MiB × (number of graphics devices) \ ③
+ (additional memory overhead) ④
```

1. Required for the processes that run in the `virt-launcher` pod.
2. Number of virtual CPUs requested by the virtual machine.
3. Number of virtual graphics cards requested by the virtual machine.
4. Additional memory overhead:
 - If your environment includes a Single Root I/O Virtualization (SR-IOV) network device or a Graphics Processing Unit (GPU), allocate 1 GiB additional memory overhead for each device.
 - If Secure Encrypted Virtualization (SEV) is enabled, add 256 MiB.
 - If Trusted Platform Module (TPM) is enabled, add 53 MiB.

Mermaid ↗

```
```mermaid
graph TD;
 A-->B;
 A-->C;
 B-->D;
 C-->D;
```
```



Markdown Preview Mermaid ↗ VSCode

WARNING
 `.md` [issue ↗](#)

```
{width="100" height="100"}
```



MDX

[MDX](#) [Markdown](#) [Markdown](#) [JSX](#) [rspress MDX](#)

rspress

doom

Overview

Directive

ExternalSite

ExternalSiteLink

AcpApisOverview ExternalApisOverview

Term

props

TermsTable

props

JsonViewer

rspress

- [Badge](#)
- [Card](#)
- [LinkCard](#)
- [PackageManagerTabs](#)
- [Steps](#)
- [Tab/Tabs](#)
- [Toc](#)

`rspress/theme`

preview.mdx

```
import { SourceCode } from '@rspress/core/theme'

<SourceCode href="/" />
```

doom

`doom`

Overview

Directive

`Directive`

- 在 ``doc/en`` 中定义 ``doc/zh`` 的文档内容

```
<Directive type="danger" title="危险">
  在 `doc/zh` 中定义文档内容
</Directive>
```

- 在 `( doc/en )` 中定义 `doc/zh` 的文档内容

危险

在 `doc/zh` 中定义文档内容

ExternalSite

在 `link.mdx` 中

```
<ExternalSite name="connectors" />
```

Note

在 `DevOps` 中定义文档内容 `DevOps` 文档内容

[DevOps 文档](#)

ExternalSiteLink

在 `link.mdx` 中

```
<ExternalSiteLink name="connectors" href="link.mdx#hash"
  children="Content" />
```

[Content](#)

TIP

mdx `<ExternalSiteLink name="connectors" href="link" children="Content" />`

```
<ExternalSiteLink name="connectors" href="link">
  Content {/* `p` ` */}
</ExternalSiteLink>
```

`p` `children`

AcpApisOverview ExternalApisOverview

API

```
<AcpApisOverview />
{/* same as following */}
<ExternalApisOverview name="acp" />

<ExternalApisOverview name="connectors" />
```

Note

ACP APIs [ACP APIs](#)

Note

DevOps APIs [DevOps APIs](#)

Term

```
<Term name="company" textCase="capitalize" />
<Term name="product" textCase="lower" />
<Term name="productShort" textCase="upper" />
```

ACP

props

- name :
- textCase : lower , upper , capitalize

TermsTable

```
<TermsTable />
```

company		-	Alauda	-	
product		-	Alauda Container Platform	-	
productShort	ACP	-	ACP	-	

props

- terms : NormalizedTermItem[]

JsonViewer

```
<JsonViewer value={{ key: 'value' }} />
```

yaml json

key: value



□□□□□□□□□□□□□□□□□□□□ shared □□□□□□□□□□□□□□□□□□□□

```
import CommonContent from './shared/CommonContent.mdx'
```

<CommonContent />

runtime [↑](#) API `.jsx/.tsx` `.mdx`

```
// shared/CommonContent.tsx

export const CommonContent = () => {
  const { page } = usePageData()
  return <div>{page.title}</div>
}

// showcase/content.mdx

import { CommonContent } from './shared/CommonContent'

<CommonContent />
```

WARNING

000000 .mdx 00000000 props 000000 issue 00000000 props 000000
 .jsx/.tsx 000000



alauda en / zh

public

en / zh



i18n.json

.ts/.tsx

.mdx

i18n.json

docs i18n.json

□□□□□□□□ useI18n □□□□□□□□□□□□□□□□

docs/i18n.json

```
{
  "title": {
    "zh": "标题",
    "en": "Title"
  },
  "description": {
    "zh": "描述",
    "en": "description"
  }
}
```

.ts/.tsx

```
import { useI18n } from '@rspress/core/runtime'

export const CommonContent = () => {
  const t = useI18n()
  return <h1>{t('title')}</h1>
}
```

.mdx

```
import { useI18n } from '@rspress/core/runtime'

# {useI18n()('title')}

{useI18n()('description')}
```

API 文档

本文档介绍了 API 文档 API 的 CRD (Custom Resource Definition) 的用法。本文档旨在帮助开发者了解 API 文档 API 的 CRD 的用法。

```
├─ apis
│   ├── advanced-apis # API
│   ├── crds # CRDs
│   └── references # 文档
```

目录

- API
- props
- CRD
- props
- 文档
- props
- openapi 文档

API 文档

advanced-apis/codeQualityTaskSummary.mdx

```
# CodeQualityTaskSummary

<OpenAPIPath path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary" />
```

文档 [CodeQualityTaskSummary](#) 文档

props

- `path` : OpenAPI schema `paths` 中的路径
- `pathPrefix` : 路径前缀 `api.pathPrefix`
- `openapiPath` : 指向 `openapi` 文档的路径

CRD

crds/ArtifactCleanupRun.mdx

```
# ArtifactCleanupRun

<K8sCrd name="artifactcleanupruns.artifacts.katanomi.dev" />
```

文档 [ArtifactCleanupRun](#) 文档

props

- `name` : CRD `metadata.name`
- `crdPath` : 指向 `openapi` 文档的路径 `CRD` 文档

文档 文档 文档 文档

references/CodeQuality.mdx

CodeQuality

```
<OpenAPIRef schema="v1alpha1.CodeQuality" />
```

□□ CodeQuality □

props

- `schema` : OpenAPI schema `definitions` (v2) or `component/schemas` (v3) □□□□
- `openapiPath` : □□□□ `openapi` □□

□□ **openapi** □□

□□ **OpenAPIPath** □ **OpenAPIRef** □□□□□□□□ **openapi** □□□□□□□□□□□□□□□□

```
□ openapi □□□□□□□ openapiPath □□□□□
```

```
<OpenAPIPath
  path="/plugins/v1alpha1/template/codeQuality/task/{task-id}/summary"
  openapiPath="shared/openapis/katanomi.json"
/>
```

权限管理

```
<K8sPermissionTable functions={['devops-testplans', 'devops-testmodules']}/>
```

权限

props

权限

props

- functions: string[] - 权限资源 FunctionResource 列表

权限

权限	权限	权限 资源	权限 资源	权限 资源	权限 资源	权限 资源	权限 资源
权限 devops- testplans	权限	✓	✓	✓	✓	✓	✗
	权限	✓	✗	✓	✓	✓	✗
	权限	✓	✗	✓	✓	✓	✗

☐ Markdown ☐☐☐☐

```
<!-- reference-end -->
```

```
{/* reference-end */}
```

```
□□□□□ <!-- reference-start#ref --> □□□□□
```

frontmatterMode

--	--	--	--	--	--

reference:

```

- repo: alauda-public/product-doc-guide # 文档仓库
  branch: # [string] 分支
  publicBase: # [string] 文档仓库根目录 /images/xx.png 图片地址
docs/public
sources:
- name: anchor # 锚点
  path: docs/index.mdx# # 文档地址
doom.config.*
  ignoreHeading: # [boolean] 是否忽略 heading true 忽略
  processors: # 处理器
    - type: ejsTemplate
      data: # ejs 模板 `<%= data.xx %>`
  frontmatterMode: merge # 处理 frontmatter 模式 ignore 忽略
ignore/merge/replace/remove

```

frontmatterMode

- **ignore** 忽略 frontmatter
- **merge** 合并 frontmatter 中的 key
- **replace** 替换 frontmatter
- **remove** 移除 frontmatter

文档地址

□□□□ >

□ □ □ □ □ □ □ □

[illegible]

--	--	--	--	--

```

$ doom build -v
$ dist -v

```

```
# 4.0 4.0
doom build -v 4.0 # 4.0 dist/4.0 {base}/4.0
doom build -v master # master dist/master {base}/master
doom build -v {other} # dist/{other} {base}/{other}

# unversioned unversioned-x.y
doom build -v unversioned # dist/unversioned {base}
doom build -v unversioned-4.0 # 4.0 dist/unversioned {base}
```

□□□□□□

```
|— console-platform
|   |— 4.0
|   |— 4.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
|— console-devops-docs
|   |— 4.0
|   |— 4.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
|— console-tekton-docs
|   |— 1.0
|   |— 1.1
|   |— index.html
|   |— overrides.yaml
|   |— versions.yaml
```

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Redirecting...</title>
    <meta http-equiv="refresh" content="0; url=/console-docs/4.1" />
  </head>
  <body>
    <p>Redirecting to <a href="/console-docs/4.1">/console-docs/4.1</a>
  </p>
  </body>
</html>
```

🌐🌐🌐🌐🌐🌐🌐🌐

overrides.yaml

```
# 🌐🌐🌐🌐🌐🌐🌐🌐🌐🌐🌐🌐🌐🌐🌐🌐
title:
  en: Doom - Alauda
  zh: Doom - 🌐🌐
logoText:
  en: Doom - Alauda
  zh: Doom - 🌐🌐
```

versions.yaml

```
- '4.1'
- '4.0'
```